

•



- [Core Concepts](#)
- Jobs and Job Profiles

On this page

Jobs and Job Profiles

Was this page helpful? 

Jobs allow Pulse to execute computational tasks in a distributed way in a Spark cluster. Job profiles are useful to fine-tune how Jobs use available resources in a Spark node. The following sections describe in more details what they are and how they work.

Jobs, Job Executions

A job is a set of computational tasks for a certain goal. For example, to run a Model Evaluation the Pulse server needs to execute a set of calculations. Those calculations are the **tasks** of that model evaluation job.

To get the result of a job faster, Pulse distributes the tasks between the nodes of a Spark cluster. Hence jobs are often referred as **Spark jobs** or as **distributed jobs**.

A job execution is an instance of a job, which is created when the job is executed. Imagine a job as a list of instructions on how to execute a complex process or computation (e. g. a Hyperparameter Tuning or a Plan Execution). The job execution, on the other hand, is an instance of a job, created when a user or a program triggers Pulse to execute those instructions on a specific set of data.

Pulse creates a job execution for the following:

- [Data Source Analysis](#)
- [Feature Importance](#)
- [Model training](#)
- [Model Evaluation](#)
- [Plan Execution](#)
- [Rules Evaluation](#)
- [Sandbox Evaluation](#)
- [Hyperparameter Tuning](#)
- [AutoML run](#)
- [Scheduled Metrics Jobs](#) (SMJs)

You can check job executions and their progress in the **Jobs** tab , under **Job Executions**.

Job Profiles

A job profile is a set of Spark configurations that control how Spark jobs behave. For example, you can use them to ensure that Spark assigns [enough memory](#) to the node where the job is executed.

You can check Job Profiles in the **Jobs** tab , under **Job Profiles**.

The scope of Job Profiles can be the following:

- **Default for the App:** applies to all jobs of the Pulse application. For each Pulse application, only one job profile can be [configured as default](#).
- **Default for a specific resource type:** each of these job profiles apply to one resource type of the following list:
 - Data Source Analysis
 - Feature Importance
 - Model Training
 - Model Evaluation
 - Plan Execution
 - Rule Snapshot
 - Sandbox Evaluation
 - Hyperparameter Tuning
 - SMJs

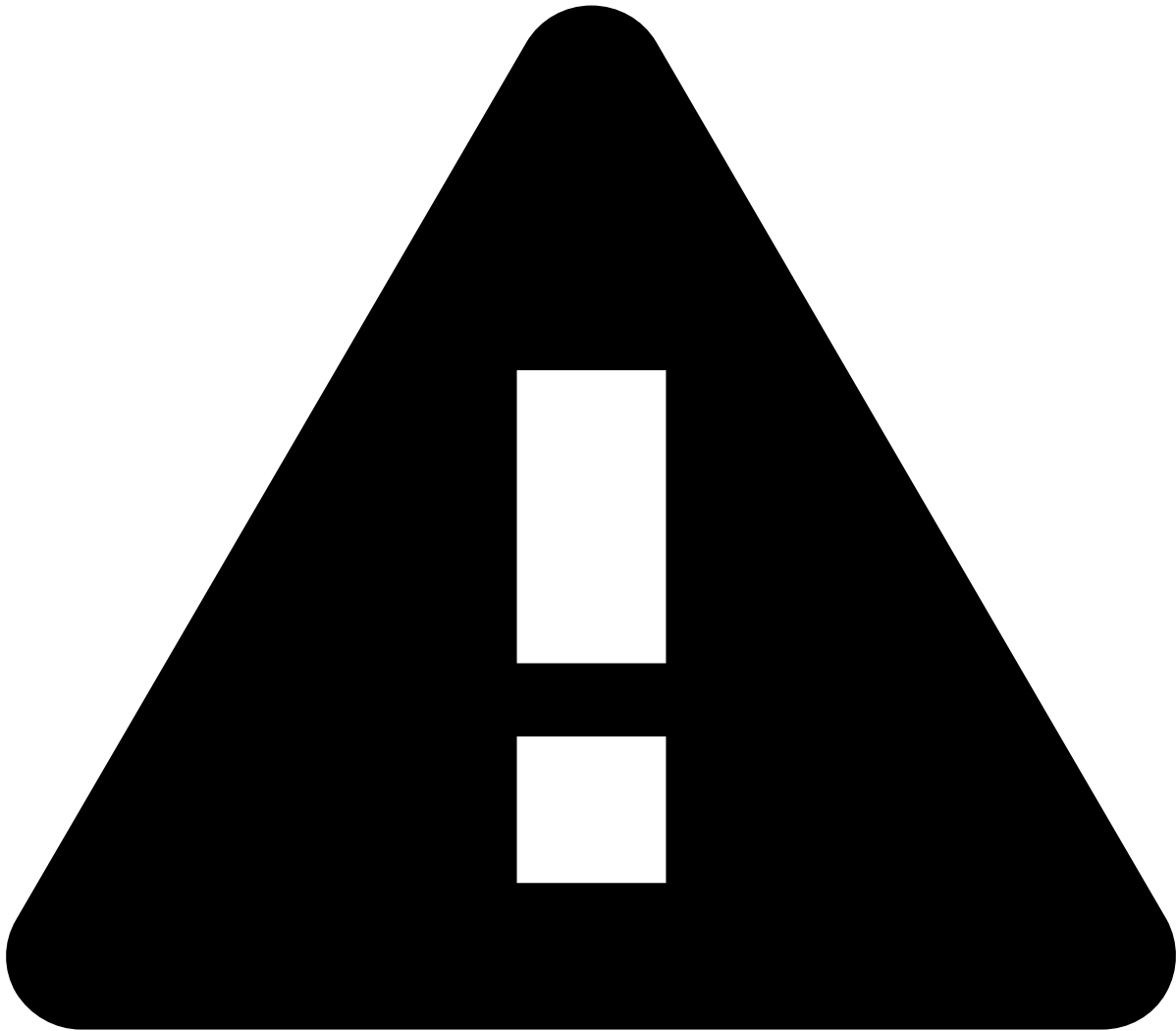


info

It's optional to have a default job profile for a resource type. If there is [a job profile configured as default for a resource type](#), for that resource it overrides the default job profile of the application.

- **Applied to a single job:** when creating a Data Science resource that can use job profiles, you can [select a job profile other than the default](#), or you can [edit the properties on demand](#), for that single job execution that you are creating.

To understand which properties were used in a job execution, open the Job Execution and select the button.

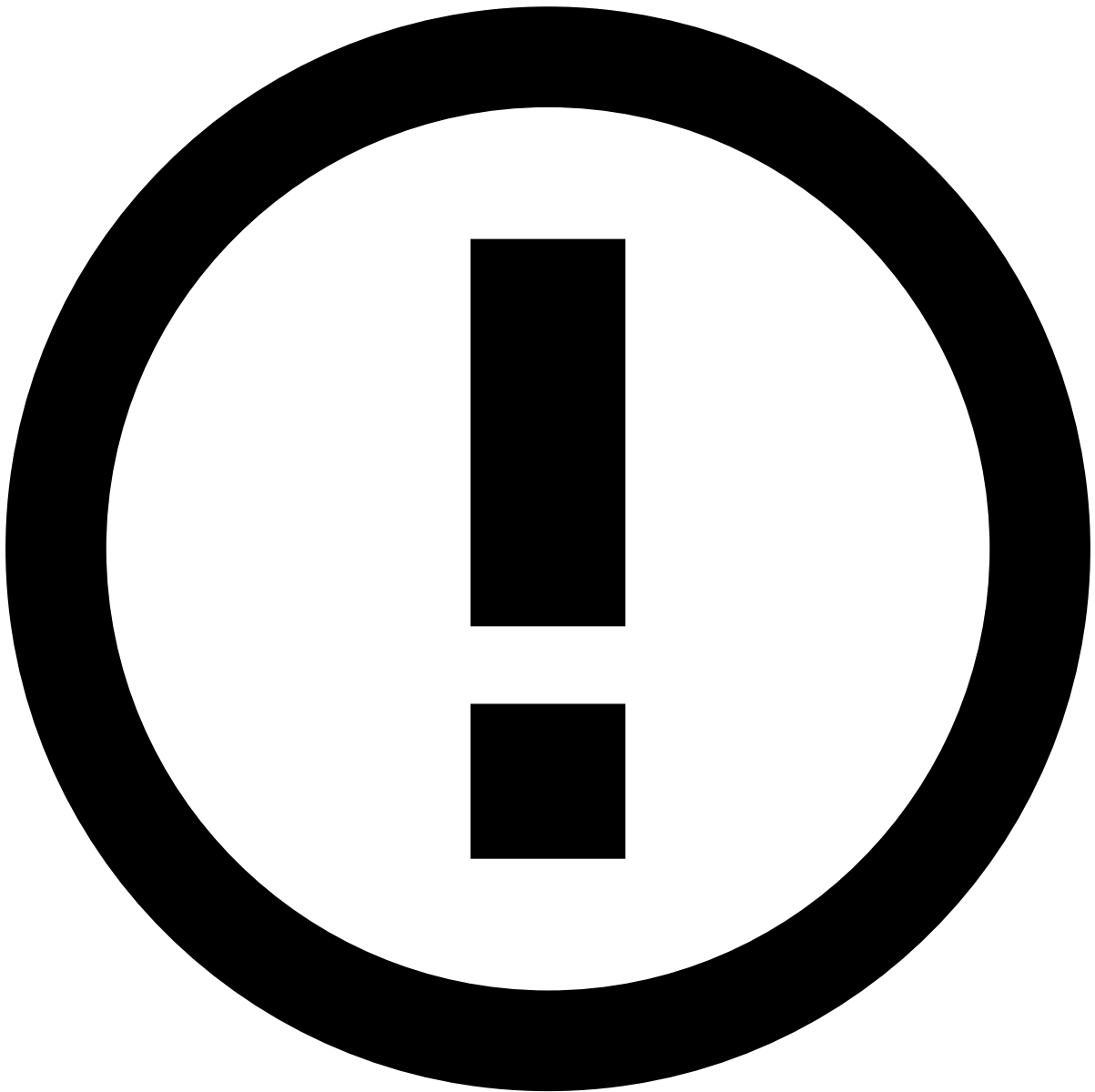


Resources without default Job Profile

- It's not possible to use job profiles with Registered Jobs and [Scheduled Metrics Jobs](#).
- Although an AutoML run creates a job, you cannot assign a default Job Profile to AutoML jobs. This is because the AutoML job will start other jobs (Plan execution, Hyperparameter tuning, etc.) which can have their default job profile.

Pulse Properties Profile

The Pulse Properties Profile (PPP) contains a read-only list of the Spark properties that are loaded from a preconfigured source.



info

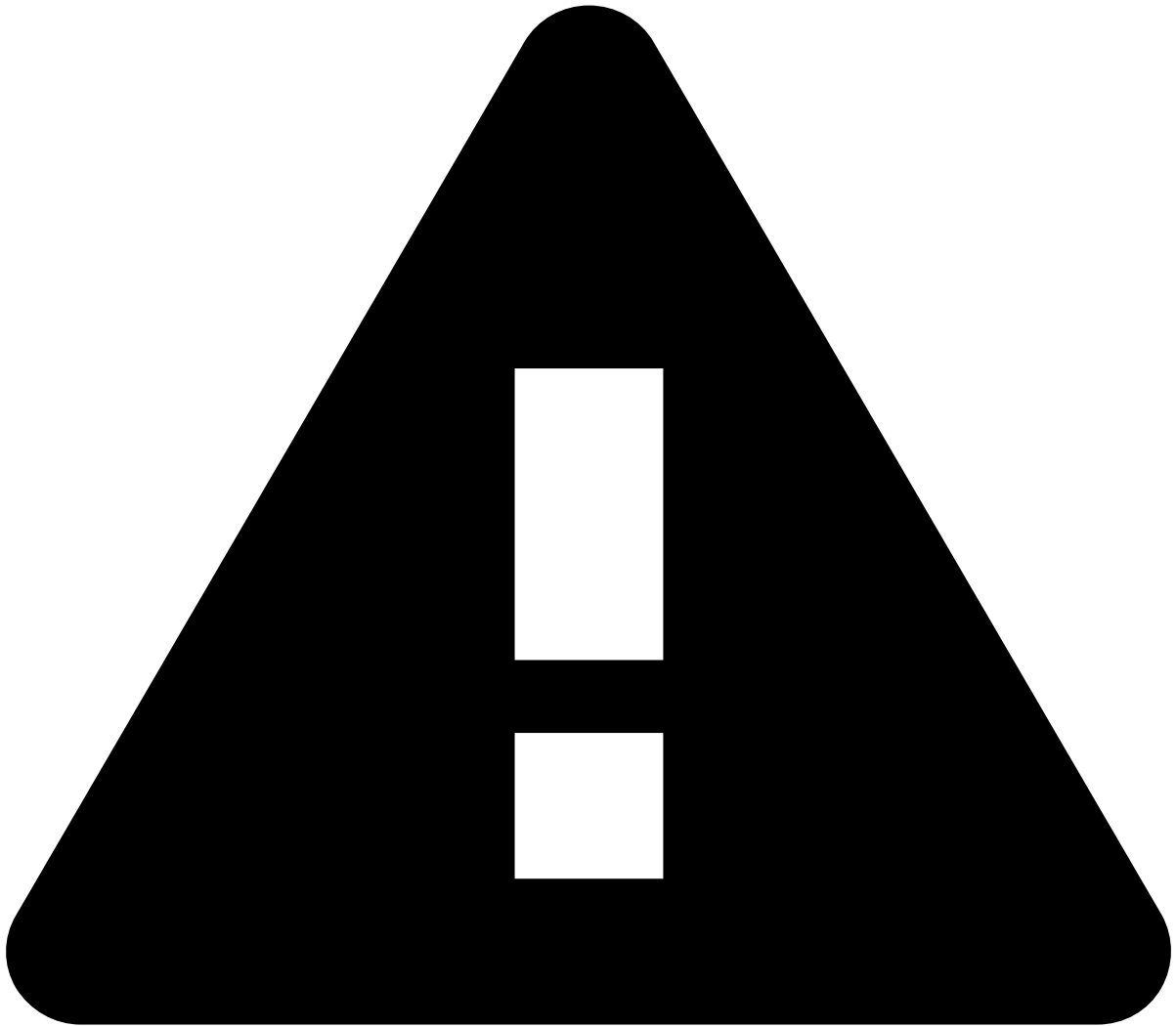
By default that source is the **pulse.properties** file located in the **conf** folder. However, the PPP can be configured to read from other types of sources, such as RDBMS, NoSQL (Cassandra), etc.

You can check the PPP by accessing the **Jobs** tab and opening the PPP in the **Job Configuration Profiles** table as shown in the following image:

When you [create a new job profile](#), the properties in the PPP are automatically inherited by this new profile. You can edit them, or add new Spark properties to the new job profile.

Updates in the PPP

When properties in the PPP are updated, those changes are also automatically propagated to the other job profiles. For example, if the PPP is updated with a new Spark property, it is also automatically included in all the existing job profiles.

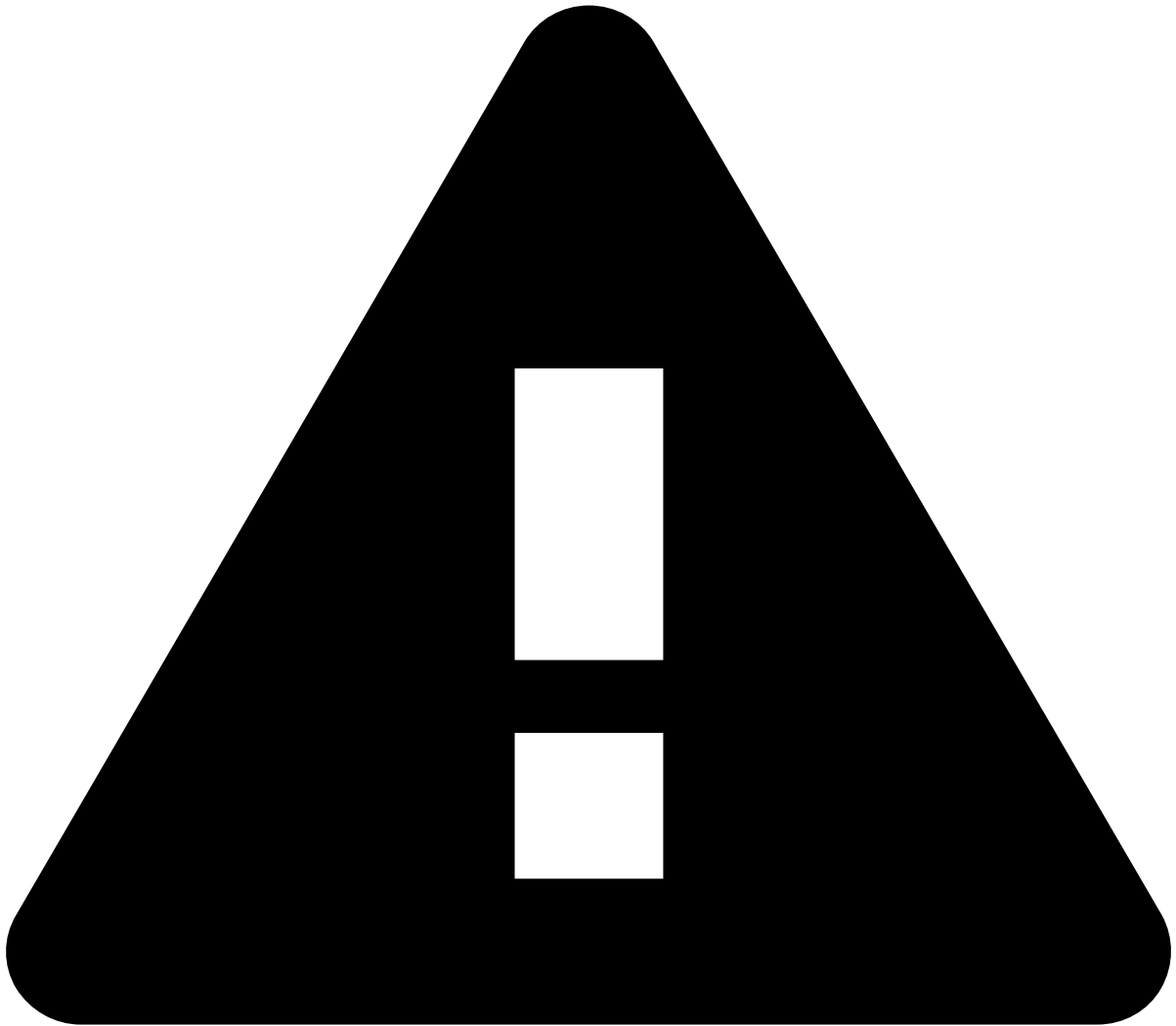


Exception: updates that are not propagated

When in the PPP you update a property which already exists in a Job Profile with a different value, the change is not populated from the PPP to the Job Profile.

Group Properties Profile

The Group Properties Profile allows specifying Spark properties at the level of Hierarchical Groups.



caution

Spark properties defined in the Group properties override those defined in the Job Profile, which in turn override those from the PPP. The order of precedence is therefore as follows: Group properties > Job Profile > PPP.



danger

This is an experimental feature that currently only works for Plan Execution jobs that employ a persistent PKernel via RocksDB.

In addition to static job profiles, we can also generate automatic configurations: this is an experimental feature that currently only works for Plan Execution jobs that employ a persistent PKernel via RocksDB.

The Spark configuration properties that are generated by the Job Profile Tuner are:

- spark.memory.fraction
- spark.memory.storageFraction
- spark.task.cores
- spark.executor.cores
- Spark.executor.memory
- spark.executor.memoryOverhead

For more information about these properties you may refer to <https://archive.apache.org/dist/spark/docs/3.3.2/configuration.html>. These properties override the ones in the default static profile.

When running a plan execution be aware that not all plans are eligible for auto-profiling. The conditions for eligibility are:

- The plan must not be omnichannel
- The plan must be a directed path graph (or linear graph); i.e., only a data source and a sink and each other operator in the plan must have at maximum one upstream operator as input and one downstream operator as output.

Under the hood, the job profile tuner (JPT), which is the component in Pulse that generates automatic profiles, uses heuristics that based on the job's input data and workload, as well as cluster availability state, attempt to generate a configuration that not only make a job viable (by executing it successfully) but also performant (minimizing latency while using resources efficiently). In a nutshell, the JPT attempts to predict the maximum memory that a Spark executor will require by assessing the artifacts that need to be maintained in memory during the job execution, such as the state of stateful operators (e.g., metrics), Pulse managed lists, and enrichers.

Since using a persistent PKernel all the times can incur a penalty in the job performance, we have a mechanism to automatically select the most appropriate state backend of the PKernel based on job characteristics, namely data distribution and window durations. To enable it, check the property `automaticStateBackend` in DataScience PKernel configurations. The state backend selection will happen before the tuning of job profiles and, therefore, a profile can only be generated if the selector picks a persistent state backend for PKernel.

To help on debugging eventual problems with the JPT, which is still an experimental feature, users may also enable the automatic collection of job performance metrics: see the property `collectPerformanceMetrics` in Spark Job Service Configuration.

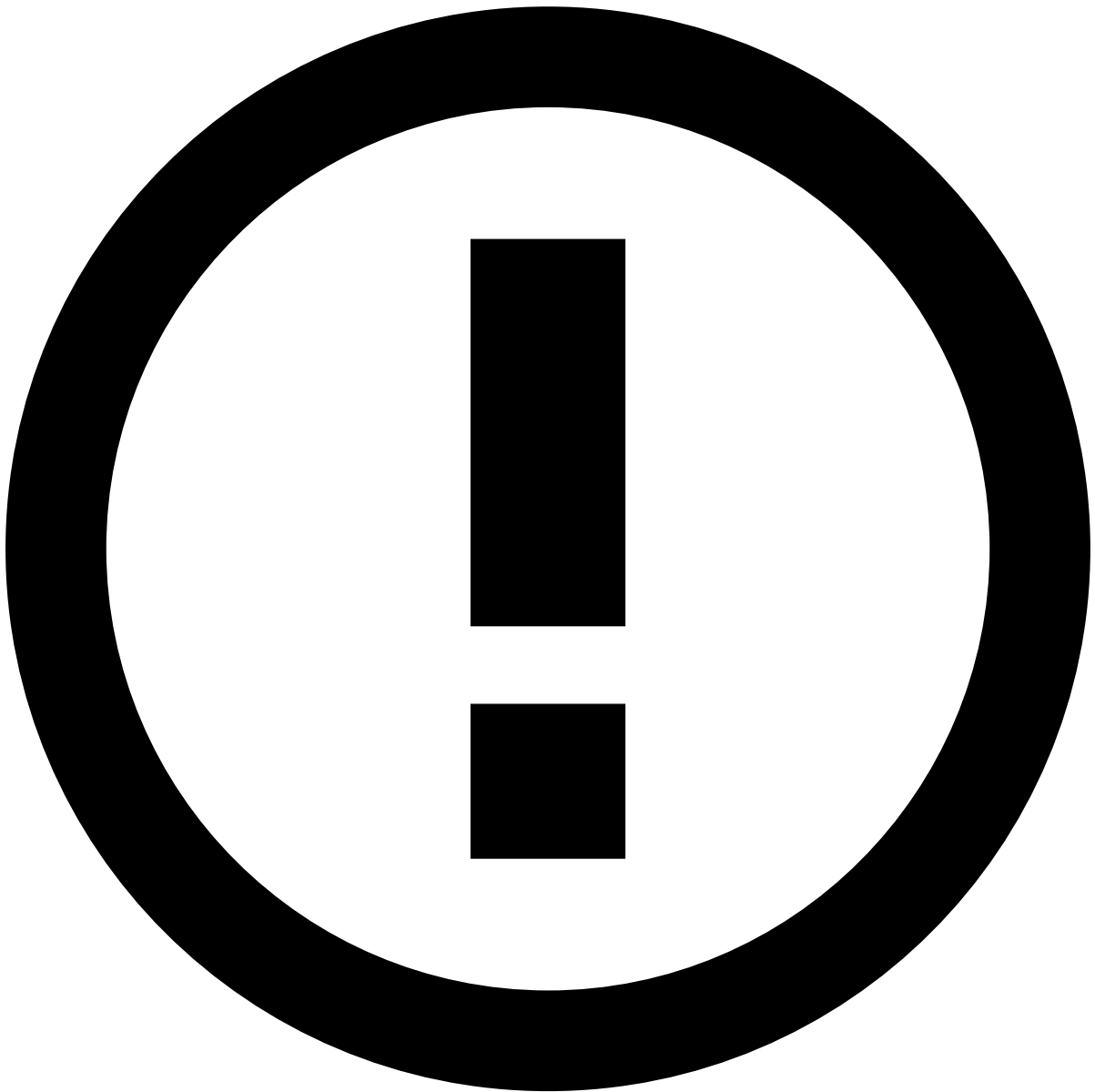
Memory considerations

When running Spark jobs, it's recommended to have a default profile for all jobs, and define job profiles for executing resource-intensive jobs.

Calculation tasks may fail if they have insufficient memory available. This might happen because each Spark node has fixed resources available, for instance a fixed number of cores and a fixed amount of memory. Some nodes may have larger resources than others, but by default it's not sure that the tasks that need large resources get assigned to those nodes. Typical examples of tasks needing larger memory are Model Trainings, Hyperparameter Tunings and Plan Executions.

If smaller jobs occupy the resources that have larger capacity, Pulse doesn't leverage the full potential of the Spark cluster. To ensure that the necessary resources are available for jobs that need them, you can use Spark properties to influence the number of cores and the amount of memory used by a job. Job profiles can regulate what kind of resources should be assigned to a job which is running with a specific job profile. For instance, you can have a default profile for all jobs, but define a job profile for executing Plans, and another one for Hyperparameter Tunings to make sure that these get the proper resources they require.

If these job profiles are created, you can use them when creating a job execution of a related Data Science resource.



Restrictive default settings

The default Spark settings are restrictive, for example, they allow 1GB of memory per node. It is recommended to create a Spark configuration strategy early on in a project, and keep adjusting the settings as you monitor the Job Executions.

Learn more

Learn how to do the following tasks related to Jobs and Job Profiles:

- [Creating a job profile](#)
- [Selecting a job profile](#)
- [Edit properties for a specific job execution on demand](#)